



SHANGHAI JIAO TONG
UNIVERSITY



数据和知识管理实验室
DATA & KNOWLEDGE MANAGEMENT LAB



When Large Language Models Meet Data Management Systems

Jiarui Jin

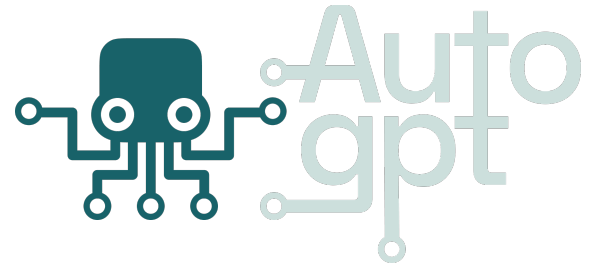
Ph.D. Student, Shanghai Jiao Tong University

<https://jinjiarui.github.io/>

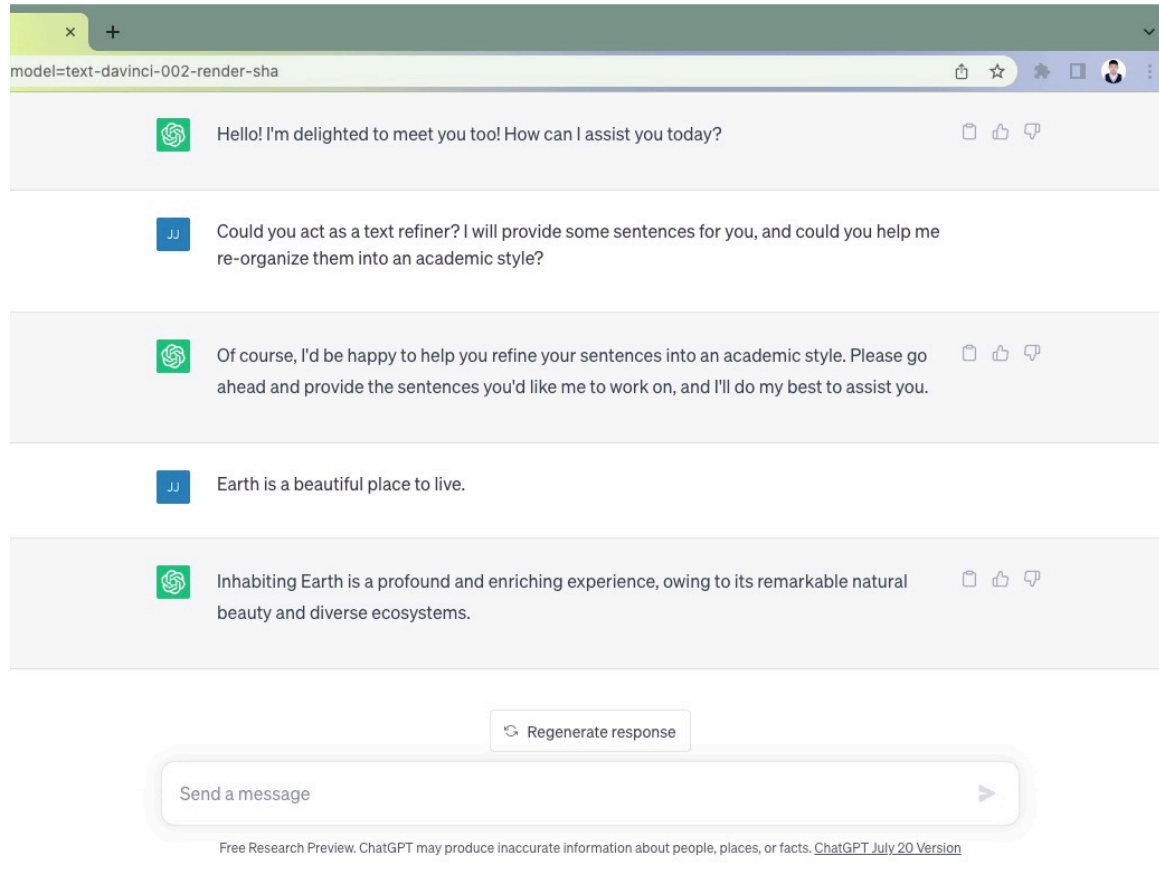
14:00-15:00, July 21, 2023
Shanghai Jiao Tong University, Shanghai

Slides: https://jinjiarui.github.io/project/files/lm4ir_sjtu.pdf

PART I: Applications and Products

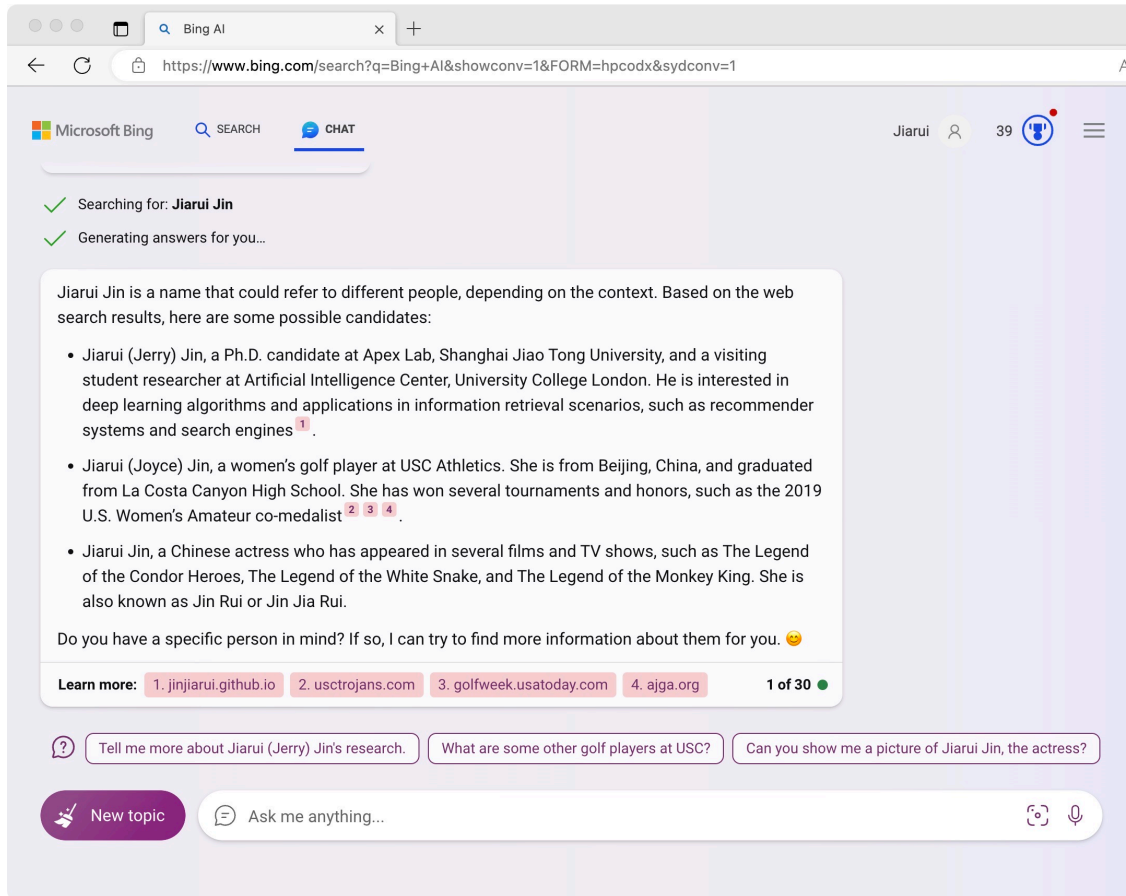


Customer Service: ChatGPT



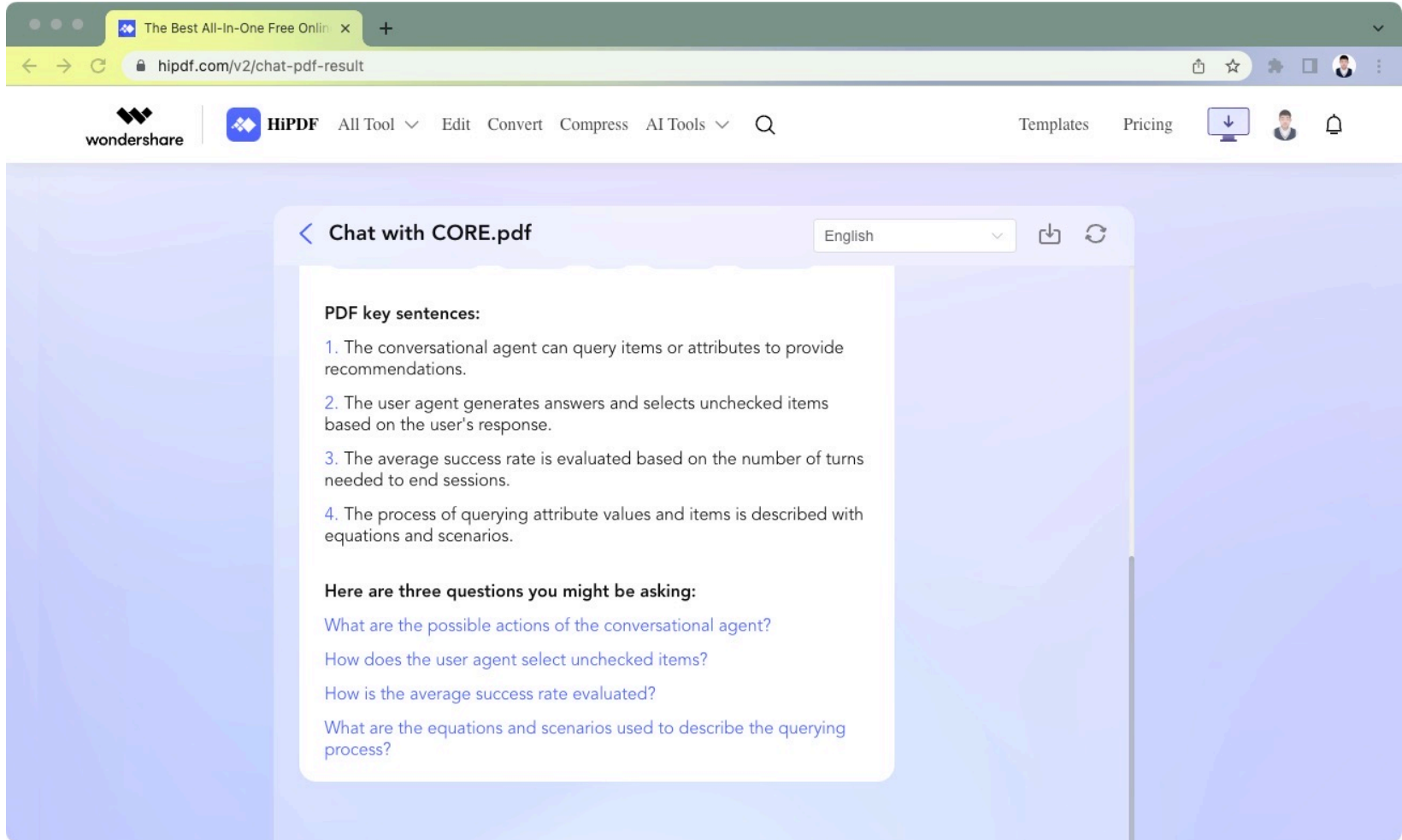
LLM can communicate like human and well do many language-related tasks such as **information extractor** or **question answer**.

Search Engine: New Bing



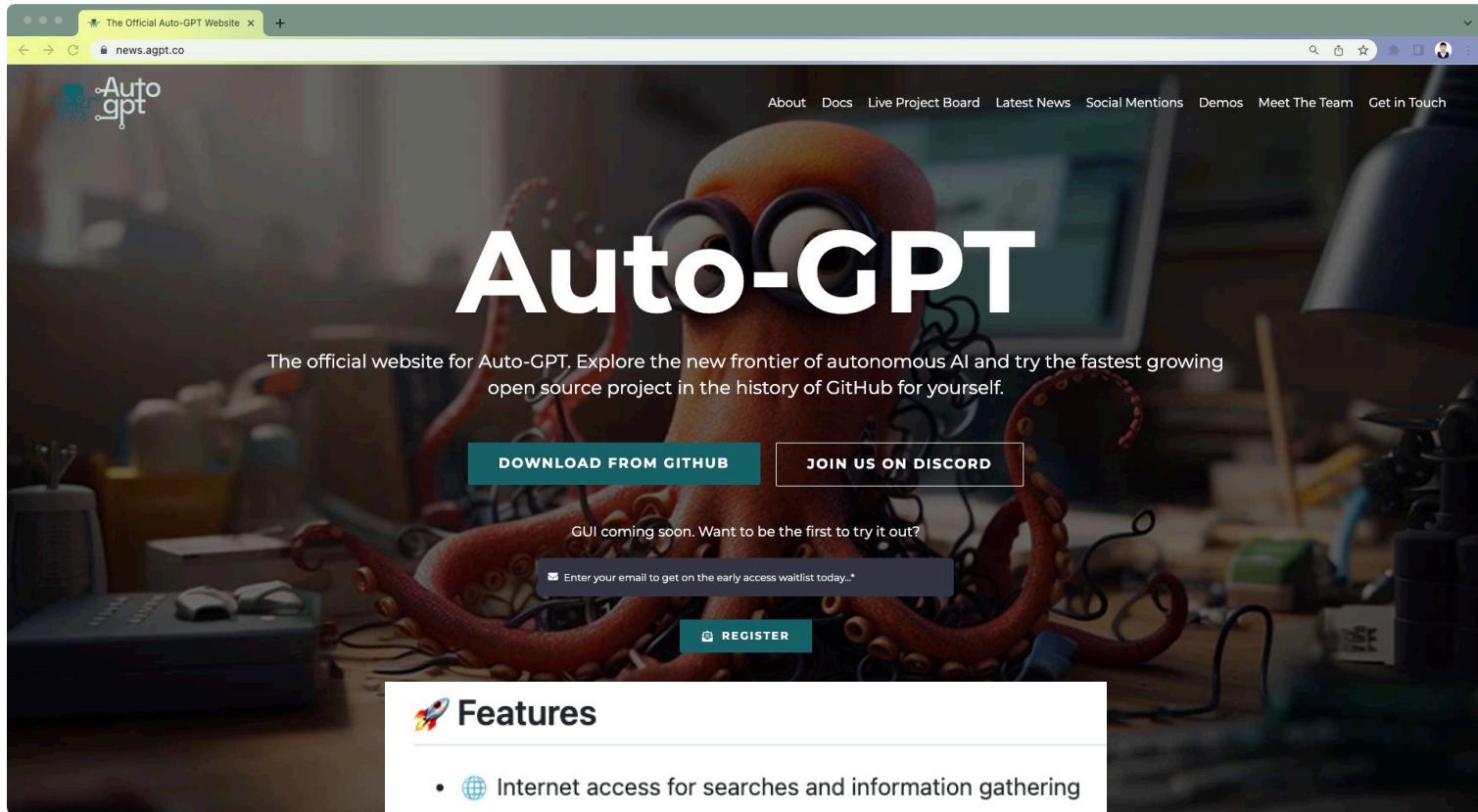
retrieve from database (or website) and organize in human-like language by LLM.

Summarize Texts: Chat PDF



extract key information and **summarize** them into human language.

Database: Vector Database



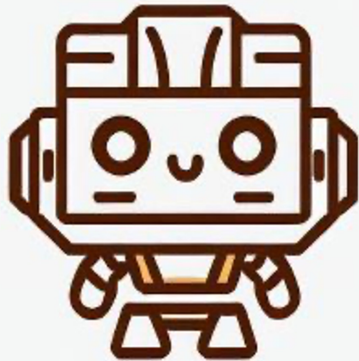
Features

- 🌐 Internet access for searches and information gathering
- 💾 Long-term and short-term memory management
- 🧠 GPT-4 instances for text generation
- 🔗 Access to popular websites and platforms
- 📁 File storage and summarization with GPT-3.5
- 🛠️ Extensibility with Plugins

Outline

- PART I: Applications and Products
- **PART II: Large Language Models are Data Filters**
- **PART III: Large Language Models are Text Renders**
- PART IV: Take-Aways

PART II: GUNDAM

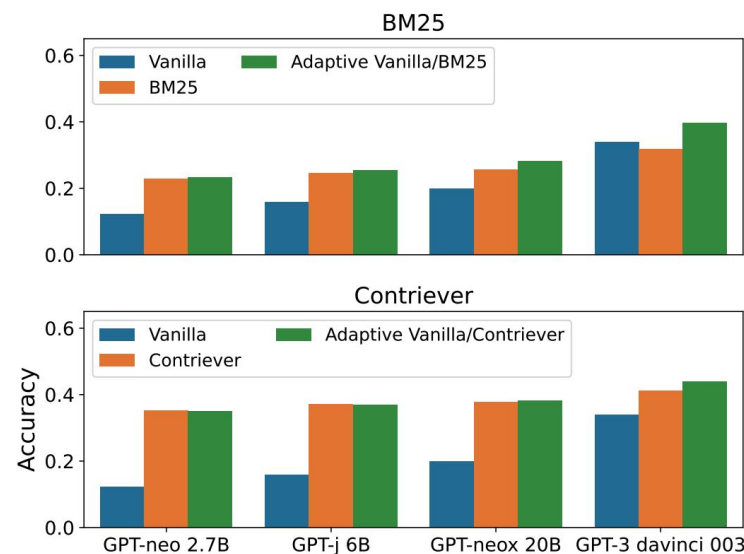
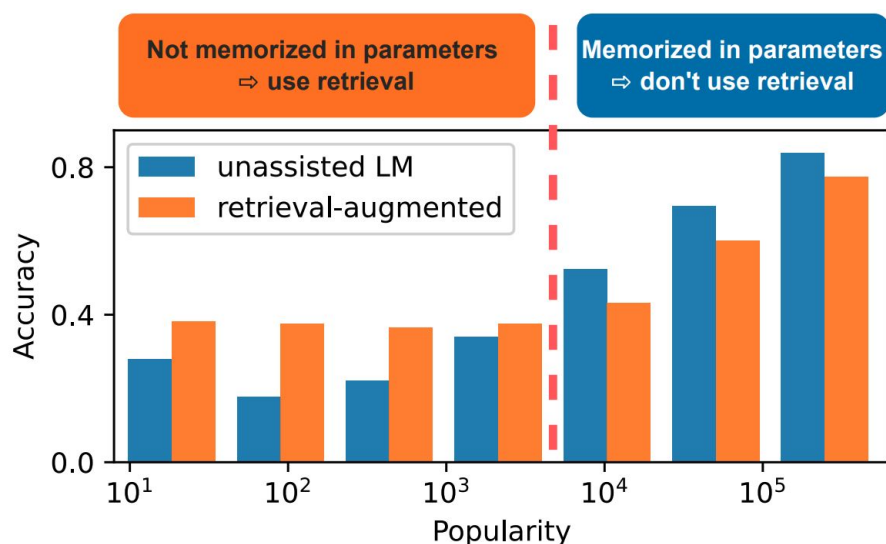


GUNDAM

Golden plUg-iN DAta Manager

Background: Some Observations

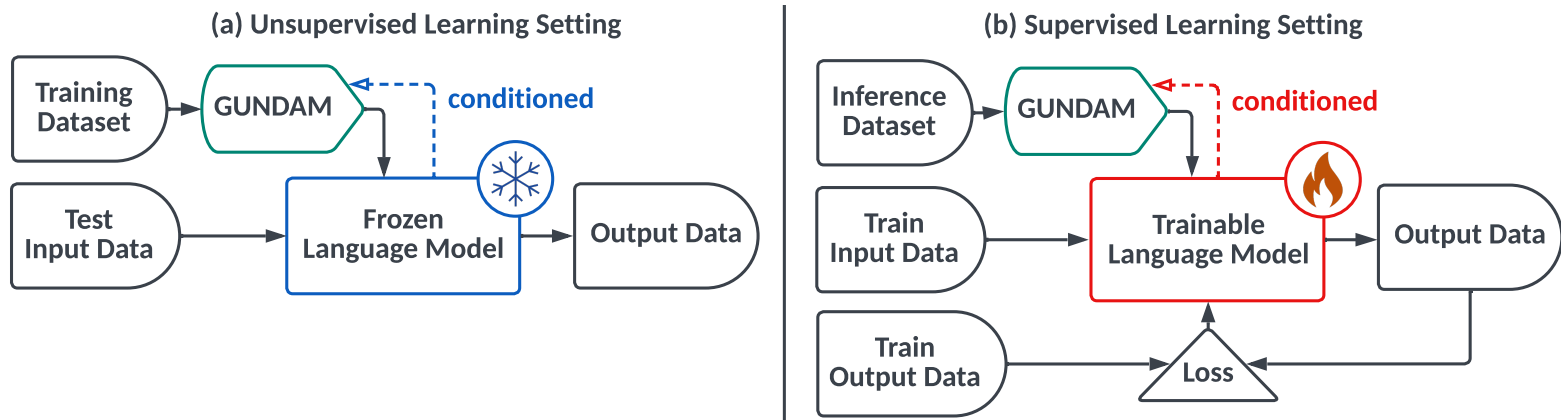
- Retrieved documents from external database or website can significantly improve the performance (i.e., accuracy and stability) of large language models, especially for questions whose popularity is lower than a threshold.



Ref: When Not to Trust Language Models: Investigating Effectiveness and Limitations of Parametric and Non-Parametric Memories. <https://arxiv.org/pdf/2212.10511.pdf>. arXiv.

Background: Some Observations

- In-context learning is using relevant documents as prompt (called demonstrations), which is an **unsupervised learning paradigm**.
- Fine-tuning always needs large corpus of textual data, but **warm-up, another supervised learning paradigm**, only pushes large language models go through the whole data corpus once.



Our Idea

- Idea is to synthesize a smaller, **golden subset $\mathcal{D}^g$** that can achieve comparable (or even better) performance of using the whole training dataset $\mathcal{D}$.
- Outer-loop: optimize $\mathcal{D}^g$ with a **frozen large language model φ**

$$\operatorname{argmin}_{\mathcal{D}^g} \mathbb{E}_{(q,a) \sim \mathcal{D}} [l(\varphi_{\mathcal{D}^g}(q), a)]$$

- Inner-loop: optimize φ for a fixed $\mathcal{D}^g$:

$$\operatorname{argmin}_{\varphi} \mathbb{E}_{(q,a) \sim \mathcal{D}^g} [l(\varphi(q), a)]$$

- This idea is actually a **data-centric perspective (data quality outweighs data quantity)** of retrieval-augmented large language model, where $\mathcal{D}^g$ is high-quality data.

What is High-Quality Data

- High-quality data should be both sufficient and necessary, where sufficiency and necessity are **conditioned on the given large language model**.

Definition 1 (Sufficient Plug-in Instance). Given tuple (X, Y, Z, S) , we say that a training example $(\mathbf{x}_n, \mathbf{y}_n)$ is a sufficient plug-in instance for a target data point $(\mathbf{x}_m, \mathbf{y}_m)$ (namely plugging-in $(\mathbf{x}_n, \mathbf{y}_n)$ is sufficient to get the right answer of $\mathbf{x}_m$), if the following equation holds:

$$Y_{\mathbf{x}_m} = 1 | \text{plug}((\mathbf{x}_n, \mathbf{y}_n)); Z = \emptyset, S = (Y_{\mathbf{x}_m} = 0). \quad (5)$$

It indicates that before plugging-in $(\mathbf{x}_n, \mathbf{y}_n)$, there is no plugged-in data (i.e., $Z = \emptyset$) and the LM's answer to $\mathbf{x}_m$ is wrong (i.e., $S = (Y_{\mathbf{x}_m} = 0)$), and when we plug-in $(\mathbf{x}_n, \mathbf{y}_n)$ (i.e., $\text{plug}((\mathbf{x}_n, \mathbf{y}_n))$), then it would correct the LM's answer to $\mathbf{x}_m$ (i.e., $Y_{\mathbf{x}_m} = 1$).

Definition 2 (Necessary Plug-in Instance). Given tuple (X, Y, Z, S) , we say that a training example $(\mathbf{x}_n, \mathbf{y}_n)$ is a necessary plug-in instance for a target data point $(\mathbf{x}_m, \mathbf{y}_m)$ (namely plugging-in $(\mathbf{x}_n, \mathbf{y}_n)$ is necessary to get the right answer of $\mathbf{x}_m$), if the following equation holds:

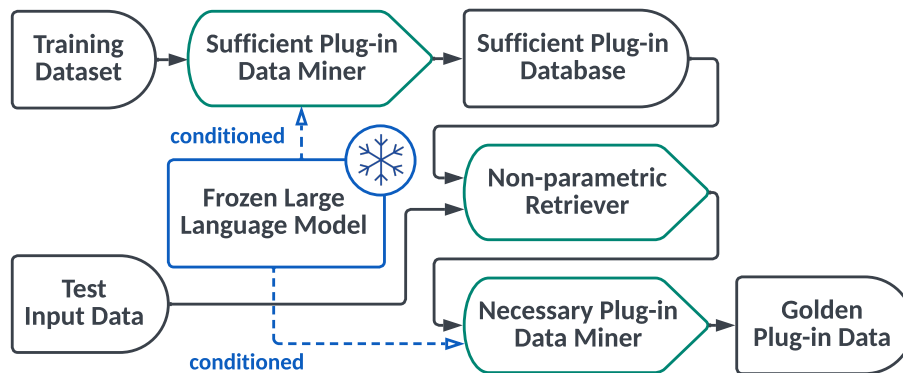
$$Y_{\mathbf{x}_m} = 0 | \text{unplug}((\mathbf{x}_n, \mathbf{y}_n)); Z = ((\mathbf{x}_n, \mathbf{y}_n)), S = (Y_{\mathbf{x}_m} = 1). \quad (6)$$

It means that before unplugging $(\mathbf{x}_n, \mathbf{y}_n)$, there is plugged-in data $(\mathbf{x}_n, \mathbf{y}_n)$ (i.e., $Z = ((\mathbf{x}_n, \mathbf{y}_n))$) and the LM's answer to $\mathbf{x}_m$ is right (i.e., $S = (Y_{\mathbf{x}_m} = 1)$), and when we unplug $(\mathbf{x}_n, \mathbf{y}_n)$ (i.e., $\text{unplug}((\mathbf{x}_n, \mathbf{y}_n))$), then it would cause the LM's answer to $\mathbf{x}_m$ being wrong (i.e., $Y_{\mathbf{x}_m} = 0$).

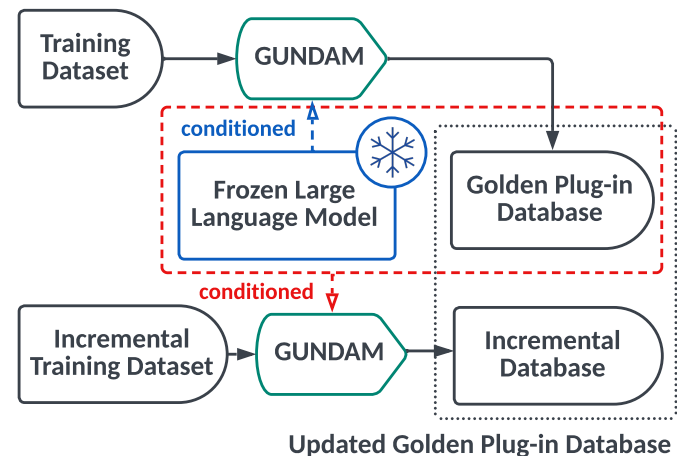
Scaling Up GUNDAM

- Adopt a **non-parametric retriever** (e.g., random, similarity-based, diversity-based retrievers) to meet the window length of large language models.
- Regard selected data and large language models as a whole to enable **incremental update**.

(a) Scaling Up GUNDAM by Incorporating it with Non-parametric Retriever



(b) Scaling Up GUNDAM by Incremental Update



Some Unsupervised Learning Results

Table 1: Result comparisons on text classification datasets for different LM bases (MED, LAR, NEO) with different retrievers (RAN, SIM, DIV) in different shot settings (1, 2, 5, 10). All the results are calculated based on 5 different random seeds, and the bold numbers mean using $\mathcal{D}_{\text{GOLD}}$ as $\tilde{\mathcal{D}}_{\text{TRAIN}}$ can obtain at least 2% improvements than using $\mathcal{D}_{\text{TRAIN}}$, or vice versa. See Table A1 for more extended results (on datasets SUBJ, and FPB).

$\Psi_{\text{LM}}(\cdot)$	$\tilde{\mathcal{D}}_{\text{TRAIN}}$	n	SST-2			SST-5			COLA			TREC		
			RAN	SIM	DIV	RAN	SIM	DIV	RAN	SIM	DIV	RAN	SIM	DIV
MED	$\mathcal{D}_{\text{TRAIN}}$	1	48.9	24.5	24.5	14.5	22.7	22.7	29.0	38.8	38.8	19.4	42.8	42.8
		2	51.2	65.7	62.5	18.0	25.6	23.7	30.9	38.5	36.2	21.4	57.2	51.4
		5	62.6	79.4	61.7	26.5	32.3	27.8	69.4	49.3	47.0	37.6	66.0	61.4
		10	50.9	83.8	76.9	14.9	35.3	30.4	31.6	52.5	58.8	53.0	71.4	65.8
	$\mathcal{D}_{\text{GOLD}}$	1	49.8	48.1	48.1	15.4	23.7	23.7	29.4	35.1	35.1	37.4	48.4	48.4
		2	67.3	67.7	64.7	20.9	27.9	25.8	31.1	41.7	34.9	27.6	56.8	52.2
		5	70.3	77.9	68.5	28.6	33.2	27.4	65.2	57.3	54.6	40.8	67.4	61.8
		10	75.2	83.0	77.2	17.6	36.2	29.8	69.3	68.7	68.5	44.6	74.6	64.6
LAR	$\mathcal{D}_{\text{TRAIN}}$	1	49.0	42.3	42.3	14.2	25.2	25.2	42.1	38.3	38.3	21.0	53.2	53.2
		2	68.0	70.7	59.6	18.1	29.7	24.4	41.1	36.8	37.7	28.2	62.6	60.6
		5	49.1	80.6	67.5	25.6	34.1	30.8	66.2	53.8	48.5	35.4	63.4	64.6
		10	71.1	84.6	73.1	28.7	38.7	36.6	43.4	55.5	56.1	43.2	66.0	68.8
	$\mathcal{D}_{\text{GOLD}}$	1	49.1	47.7	47.7	18.7	25.5	25.5	49.6	35.1	35.1	17.4	52.6	52.6
		2	67.8	73.0	61.2	25.2	29.7	24.1	38.1	36.6	37.0	34.6	62.2	59.4
		5	59.3	80.9	69.6	39.3	35.2	31.0	69.2	68.6	66.6	45.4	65.5	63.7
		10	76.0	84.7	75.6	39.6	38.8	37.1	69.3	68.8	68.9	55.8	70.4	68.6
NEO	$\mathcal{D}_{\text{TRAIN}}$	1	49.2	33.8	33.8	12.8	20.2	20.2	25.5	36.5	36.5	11.0	57.2	57.2
		2	76.8	81.5	76.3	17.9	26.9	22.7	30.7	55.5	56.5	17.6	52.6	42.2
		5	65.1	80.8	66.1	19.0	29.2	25.1	40.0	55.9	52.5	25.2	66.4	61.8
		10	69.8	84.1	69.7	12.7	33.7	31.9	69.6	59.3	63.4	41.6	70.6	69.0
	$\mathcal{D}_{\text{GOLD}}$	1	49.3	48.3	48.3	18.5	20.6	20.6	48.3	34.8	34.8	18.2	56.4	56.4
		2	75.1	82.6	78.5	19.7	27.4	22.8	69.3	64.7	64.7	27.8	54.0	44.5
		5	73.2	82.9	71.6	19.2	30.2	26.4	68.7	67.2	65.8	50.4	66.0	61.6
		10	72.4	85.8	71.8	15.4	37.0	34.5	69.8	68.8	68.9	45.2	72.8	68.8

Ref: Manage Your Plug-in Data for Language Models: A Data-Centric Approach.

<https://jinjiarui.github.io/preprints/GUNDAM.pdf>. Pre-print.

Some Supervised Learning Results

Table A3: Result comparisons on prompt-based fine-tuning the LM by using the golden plug-in data as the reference data (as shown in the black part in Algorithm A5) with different retrievers (RAN, SIM, DIV) in different shot settings (1, 2, 5, 10). For comparison, we also report the results of using the whole training dataset as the reference data. All the results are calculated based on 5 different random seeds, and the bold numbers mean using $\mathcal{D}_{\text{GOLD}}$ as $\tilde{\mathcal{D}}_{\text{REFER}}$ can obtain at least 2% improvements than using $\mathcal{D}_{\text{TRAIN}}$.

$\Psi_{\text{LM}}(\cdot)$	$\tilde{\mathcal{D}}_{\text{REFER}}$	n	COLA		
			RAN	SIM	DIV
NEO	$\mathcal{D}_{\text{TRAIN}}$	1	49.3	68.5	68.5
		2	70.8	63.9	64.3
		5	30.7	54.4	54.0
		10	30.7	50.7	47.6
	$\mathcal{D}_{\text{GOLD}}$	1	76.8	73.4	72.7
		2	74.1	71.3	71.5
		5	71.7	69.4	70.0
		10	71.3	68.5	68.5

Ref: Manage Your Plug-in Data for Language Models: A Data-Centric Approach.
<https://jinjiarui.github.io/preprints/GUNDAM.pdf>. Pre-print.

Open-Source System

- Now we support GPT-2 (medium, large, neo 1B) bases.
- Still a lot of work to do :P Expect to be done in September.

The screenshot displays the GitHub repository page for `GUNDAM-Labet/GUNDAM`. The repository is owned by `Jinjiarui` and is in a `minor` state. It has 640a35b commit, 2 days ago, and 49 commits.

Repository Structure:

File/Folder	Path	Last Commit
assets	readme	last month
docs	debug	2 weeks ago
system	minor	2 days ago
.gitignore	minor	last week
LICENSE	debug	2 weeks ago
README.md	docs	2 weeks ago
requirements.yml	requirements	last month

README.md Content:

 **GUNDAM**
Golden plUg-iN DAta Manager

Golden plUg-iN DAta Manager (GUNDAM)

GUNDAM is an open data manager based on large language models.

Repository Statistics:

- No description, website, or topics provided.
- Readme
- Apache-2.0 license
- Activity
- 53 stars
- 0 watching
- 0 forks
- Report repository

Releases:

- No releases published
- [Create a new release](#)

Packages:

- No packages published
- [Publish your first package](#)

Languages:

- Python 100.0%

Suggested Workflows:

Based on your tech stack

[Actions Importer](#) [Set up](#)

Automatically convert C/CD files to YAML for GitHub Actions.

Case Study

We consider the factual error made by Google Bard in the first demo². Bard said *The James Webb Space Telescope took the very first pictures of a planet outside of our own solar system*. However, the fact is *The Very Large Telescope took the very first pictures of a planet outside of our own solar system*. Based on the above statements, we produce the question *What took the very first pictures of a planet outside of our own solar system?*

Then, we ask the question to gpt-3.5-turbo. The corresponding answer is:

The first pictures of a planet outside of our own solar system were taken by the Hubble Space Telescope in 2008. The planet, named Fomalhaut b, orbits the star Fomalhaut, which is located about 25 light-years away from Earth in the constellation Piscis Austrinus.

We can see that the above answer is wrong. We then prompt the fact into the LM, then the answer is:

The Very Large Telescope took the very first pictures of a planet outside of our own solar system.

Case Study

The above answer is right, which verifies our Assumption A1. We ask gpt-3.5-turbo to generate some sufficient statements for the fact. The following is its output:

1. The Very Large Telescope made history by capturing the first-ever images of a planet beyond our solar system.
2. Thanks to the advanced technology of the Very Large Telescope, scientists were able to capture the first-ever photographs of an exoplanet.
3. The groundbreaking images captured by the Very Large Telescope marked a major milestone in the study of exoplanets.
4. The discovery of a planet outside of our solar system was made possible by the cutting-edge capabilities of the Very Large Telescope.
5. The Very Large Telescope's ability to capture images of distant celestial bodies allowed scientists to observe an exoplanet for the first time in history.

Case Study

We also evaluate separately using the above statements as a prompt to gpt-3.5-turbo, and either one of them can lead to a right answer. We provide detailed answers as follows.

1. The Very Large Telescope took the very first pictures of a planet outside of our own solar system.
2. The Very Large Telescope took the very first pictures of a planet outside of our own solar system.
3. The Very Large Telescope took the very first pictures of a planet outside of our own solar system.
4. The Very Large Telescope took the very first pictures of a planet outside of our own solar system.
5. The Very Large Telescope took the very first pictures of a planet outside of our own solar system.

We then prompt the above five statements into gpt-j-6b, then the answer is:

1. The first-ever images of a planet beyond our solar system have been captured by the Very Large Telescope in Chile.
2. The Hubble Space Telescope.
3. A team of astronomers led by the University of Arizona's Michael Liu.
4. The Hubble Space Telescope.
5. The Very Large Telescope.

Ref: Manage Your Plug-in Data for Language Models: A Data-Centric Approach.
<https://jinjiarui.github.io/preprints/GUNDAM.pdf>. Pre-print.

PART III: CORE



CORE

A Plug-and-Play COnversational
Agent for REcommender System

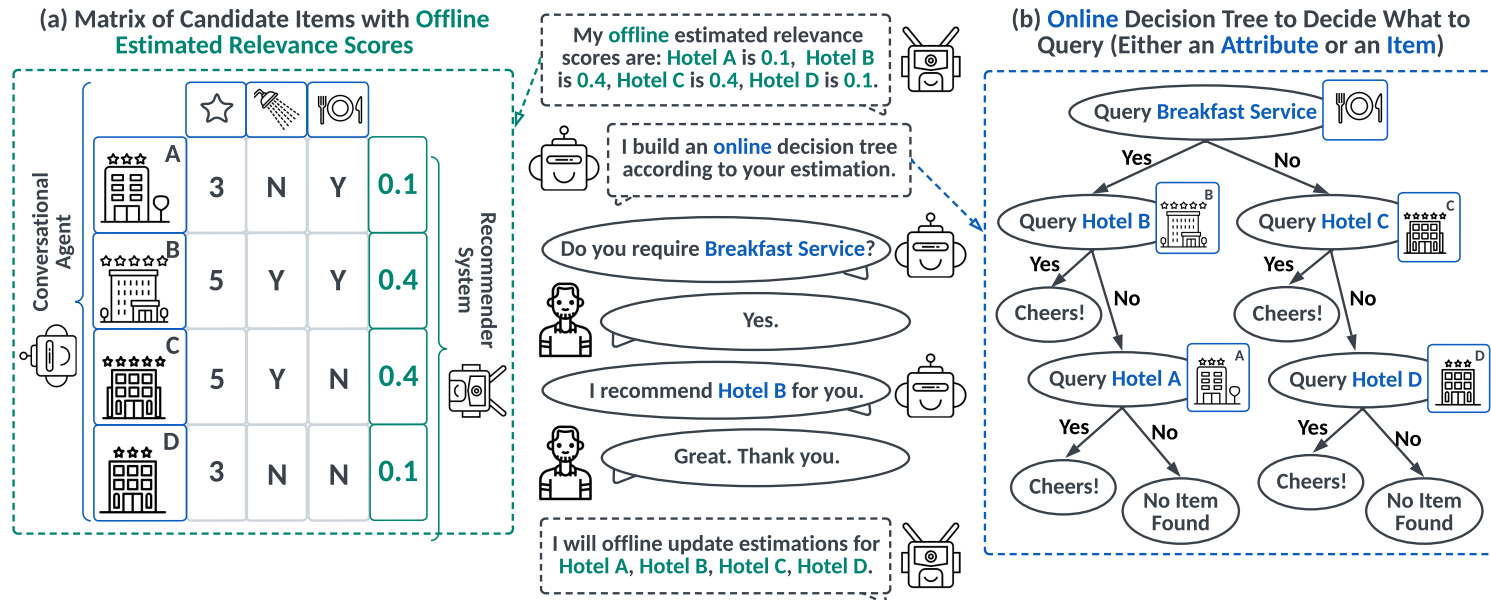
Background

- Conversational recommender systems are designed to online query user preference on **attributes and items**.
- Existing solutions are bridging attribute recommendation and item recommendation components and conversational component into a whole system through a **reinforcement learning** framework.
- Drawback 1: People may not know her preference on an attribute, but she may know whether she likes a specific **attribute value**.
- Drawback 2: Many powerful large language models can be only access through **APIs**, can not be end-to-end optimized.

Our Idea

- Bridge recommender systems and conversational agents through an offline-training and online-checking framework, only requiring **estimated scores** from recommender systems and **APIs** from conversational agent.

An Example of CORE: an **Offline-training** and **Online-checking** Framework



Online Decision Tree

- Metric: how many **relevance scores** have been checked by querying an item or an attribute.

Definition 1 (Uncertainty and Certainty Gain). For the k -th turn, we define uncertainty, denoted as U_k , to measure how many estimated relevance scores are still unchecked, i.e.,

$$U_k := \text{SUM}(\{\Psi_{\text{RE}}(v_m) | v_m \in \mathcal{V}_k\}), \quad (1)$$

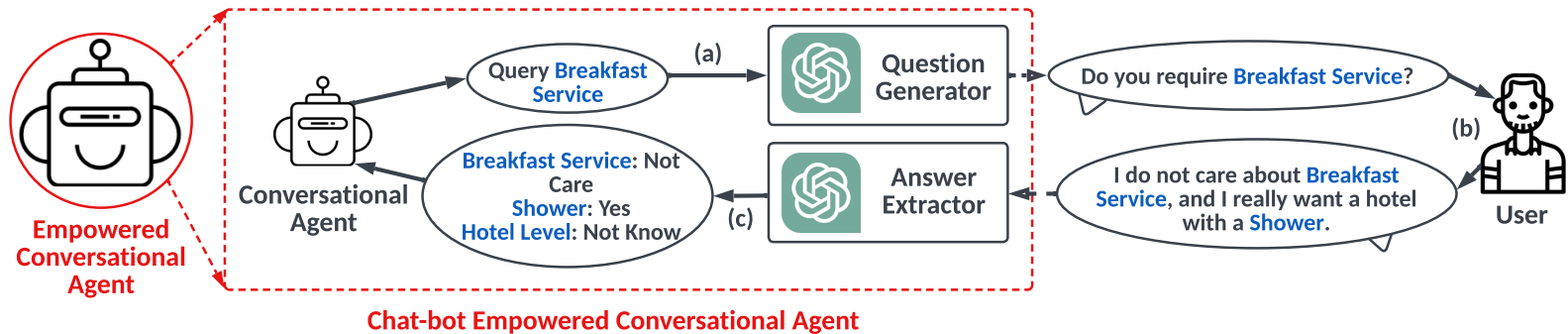
where $\Psi_{\text{RE}}(v_m)$ ¹ outputs the estimated relevance score for item v_m , $\mathcal{V}_k$ is the set of all the unchecked items after k interactions, and $\mathcal{V}_k$ is initialized as $\mathcal{V}_0 = \mathcal{V}$. Then, the certainty gain of k -th interaction is defined as $\Delta U_k := U_{k-1} - U_k$, i.e., how many relevance scores are checked at the k -th turn. Since our goal is to find a target item, if $\Psi_{\text{CO}}(\cdot)$ successfully finds one at the k -th turn, then we set all the items checked, namely $U_k = 0$.

In this regard, our conversational agent is to minimize U_k at each k -th turn via removing those checked items from $\mathcal{V}_k$. Considering that online updating $\Psi_{\text{RE}}(\cdot)$ is infeasible in practice due to the high latency and computation costs, the objective of $\Psi_{\text{CO}}(\cdot)$ can be expressed as:

$$\min_{\Psi_{\text{RE}}^*} K, \text{ s.t., } U_K = 0, \quad (2)$$

where K is the number of turns, and Ψ_{RE}^* means that the recommender system is frozen. To this end, the design of our conversational agent could be organized as an uncertainty minimization problem.

Prompt Design for Extractor and Generator



Ref: Lending Interaction Wings to Recommender Systems with Plug-and-Play Conversational Agents.
<https://jinjiarui.github.io/preprints/CORE.pdf>. Pre-print.

Prompt Design for Extractor and Generator

```
"""
prompt = f"""
You will be provided with text delimited by triple quotes.
If you can infer the user preference on attributes,
then re-write the text in the following format:
[attribute name]: [user preference]
Attribute names include Breakfast Service, Hotel Level, and Shower.
User preference includes Yes to denote the positive preference, No to denote the
negative preference, and Not Care to denote the user does not care.
If you can not infer the user preference on attributes,
then re-write the text in the following format:
[attribute name]: Not Know
'''{text}'''
"""

response = get_completion(prompt)
print(response)
```

The following is the corresponding output by the LLM.

```
Breakfast Service: Not Care
Hotel Level: Not Know
Shower: Yes
```

Prompt Design for Extractor and Generator

```
# Large language model as question generator.
text = f"""
Attribute, Hotel Level is 5, Hotel
"""

prompt = f"""
You will be provided with text delimited by triple quotes.
If it starts with the word "item", it denotes an item \
then you should generate a text to recommend the item to the user.
Otherwise, it denotes an attribute \
then you should generate a text to query the user's preference on the attribute.
You should be gentle.
'''{text}'''
"""

response = get_completion(prompt)
print(response)
```

The following is the corresponding output by the LLM.

Excuse me, may I ask for your preference on hotel level? Would you prefer a 5-star hotel or are you open to other options?

Some Results

Table 2: Results comparison of querying attribute values on tabular datasets. See Table A2 for the full version.

$\Psi_{RE}(\cdot)$	$\Psi_{CO}(\cdot)$	Amazon				LastFM				Yelp			
		T@3	S@3	T@5	S@5	T@3	S@3	T@5	S@5	T@3	S@3	T@5	S@5
COLD START	AG	6.47	0.12	7.83	0.23	6.77	0.05	8.32	0.14	6.65	0.08	8.29	0.13
	ME	6.50	0.12	8.34	0.16	6.84	0.04	8.56	0.11	6.40	0.15	8.18	0.20
	CORE	6.02	0.25	6.18	0.65	5.84	0.29	5.72	0.74	5.25	0.19	6.23	0.65
	CORE _D ⁺	6.00	0.26	6.01	0.67	5.79	0.30	5.70	0.75	5.02	0.21	6.12	0.68
FM	AG	2.76	0.74	2.97	0.83	4.14	0.52	4.67	0.64	3.29	0.70	3.39	0.81
	CRM	4.58	0.28	6.42	0.38	4.23	0.34	5.87	0.63	4.12	0.25	6.01	0.69
	EAR	4.13	0.32	6.32	0.42	4.02	0.38	5.45	0.67	4.10	0.28	5.95	0.72
	CORE	3.26	0.83	3.19	0.99	3.79	0.72	3.50	0.99	3.14	0.84	3.20	0.99
DEEP FM	CORE _D ⁺	3.16	0.85	3.22	1.00	3.75	0.74	3.53	1.00	3.10	0.85	3.23	1.00
	AG	3.07	0.71	3.27	0.82	3.50	0.68	3.84	0.79	3.09	0.74	3.11	0.88
	CRM	4.51	0.29	6.32	0.40	4.18	0.38	5.88	0.63	4.11	0.23	6.02	0.71
	EAR	4.47	0.30	6.35	0.43	4.01	0.37	5.43	0.69	4.01	0.32	5.74	0.75
PNN	CORE	3.23	0.85	3.22	0.99	3.47	0.81	3.34	1.00	2.98	0.93	3.11	1.00
	AG	3.02	0.74	3.10	0.87	3.44	0.67	3.53	0.84	2.83	0.77	2.82	0.91
	CORE	3.01	0.88	3.04	0.99	3.10	0.87	3.20	0.99	2.75	0.88	2.76	1.00

Ref: Lending Interaction Wings to Recommender Systems with Plug-and-Play Conversational Agents.
<https://jinjiarui.github.io/preprints/CORE.pdf>. Pre-print.

Open-Source System

- Now we support top-N recommendations for items and attribute values.
- Still a lot of work to do :P Expect to be done in September.

The screenshot displays the GitHub repository page for `CORE-Labet/CORE`. The repository is owned by `Jinjiarui` and has 9837763 commits, 2 days ago, with 44 commits. The repository structure is as follows:

File/Folder	Commit Message	Commit Time
assets	readme	last month
docs	doc init	2 weeks ago
system	minor	2 days ago
.gitignore	debug	2 weeks ago
LICENSE	debug	2 weeks ago
README.md	doc	2 weeks ago

The README.md file contains the following content:

CORE
A Plug-and-Play **CON**versational Agent for **RE**commender System

Conversational Agent for Recommender System

CORE is an open plug-and-play conversational agent based on large language models that can be applied to any recommender system.

To run docs

The right sidebar shows repository statistics: No description, website, or topics provided. Readme, Apache-2.0 license, Activity, 54 stars, 0 watching, 0 forks, and Report repository. The Releases section shows No releases published and a link to Create a new release. The Packages section shows No packages published and a link to Publish your first package. The Languages section shows Python 100.0%. The Suggested Workflows section shows Actions Importer with a Set up button.

PART IV: Take-Aways

Summary

- **GUNDAM**: large language models can **store** and **filter** data.
- **CORE**: using APIs of large language models can make **communication agents** (to lend **online wings** to an offline algorithm).
- Let **study** and **use** large language models as **pets**.



SHANGHAI JIAO TONG
UNIVERSITY



数据和知识管理实验室
DATA & KNOWLEDGE MANAGEMENT LAB



Thanks for Your Listening

Email: jnjarui97@gmail.com

Slides: https://jnjarui.github.io/project/files/lm4ir_sjtu.pdf